Le jeu des allumettes

Conditions, boucles et interaction en Python

Chaque partie existe en trois niveaux : la version **brute** ne donne que l'énoncé, la version **guidée** découpe le travail en étapes avec un squelette à compléter, et la version **ultra guidée** construit le programme pas à pas avec un test à chaque étape. Choisis ton niveau, et n'hésite pas à changer en cours de route !

Sommaire

Clique sur une ligne pour aller directement à la partie.

Partie 1 : version console

Version brute	énoncé seul	2
Version guidée	étapes + squelette	3
Version ultra guidée	pas à pas, tests, pièges	6

Partie 2 : version graphique (nsi_ui)

Version brute	énoncé seul	11
Version guidée	étapes + squelette	12
Version ultra guidée	pas à pas, tests, pièges	14

Partie 3 : variante graphique avec try / except

Champ de saisie et gestion des erreurs	étapes + squelette	19

Le jeu des allumettes

Partie 1 : version console (brute)

Règle du jeu

Il y a **20 allumettes** sur la table. Le joueur et l'ordinateur jouent chacun leur tour, en commençant par le joueur. À chaque tour, on retire 1, 2 ou 3 allumettes. **Celui qui prend la dernière allumette a perdu.**

Travail demandé

Écris un programme Python qui :

- affiche le nombre d'allumettes restantes à chaque tour, à l'aide d'une chaîne de formatage ;
- demande au joueur combien il en prend, et redemande si son choix est interdit ;
- ne s'arrête pas si le joueur tape autre chose qu'un nombre (utilise `try` / `except`) ;
- fait jouer l'ordinateur avec la règle suivante : il prend `(allumettes - 1) % 4` allumettes, ou 1 si ce calcul donne 0 ;
- annonce le gagnant à la fin.

Question bonus

L'ordinateur suit une stratégie. Combien d'allumettes le joueur doit-il prendre au premier tour pour être sûr de gagner ?

Le jeu des allumettes

Partie 1 : version console (guidée)

Règle du jeu

Il y a **20 allumettes** sur la table. Le joueur et l'ordinateur jouent chacun leur tour, en commençant par le joueur. À chaque tour, on retire 1, 2 ou 3 allumettes. **Celui qui prend la dernière allumette a perdu.**

Étape 1 : les variables

Crée une variable `allumettes` qui vaut 20, et une variable `tour_joueur` qui vaut `True` (elle indique si c'est au joueur de jouer).

Étape 2 : la boucle de jeu

Le jeu continue tant qu'il reste des allumettes. Quelle boucle utiliser ? Avec quelle condition ? Au début de chaque tour, affiche le nombre d'allumettes restantes avec une chaîne de formatage, par exemple `f"Il reste {allumettes} allumettes"`.

Étape 3 : le tour du joueur

Si c'est au joueur de jouer, demande-lui un nombre avec `int(input(...))`. Ce nombre est interdit s'il est plus petit que 1, plus grand que 3, ou plus grand que le nombre d'allumettes restantes. Tant que la saisie n'est pas correcte, redemande.

Étape 4 : les erreurs de saisie avec try / except

Le problème. Si le joueur tape « trois » au lieu de 3, la conversion `int()` échoue. Python provoque alors une erreur de type `ValueError` et le programme s'arrête net :

```
Combien en prends-tu (1 à 3) ? trois
Traceback (most recent call last):
  ...
ValueError: invalid literal for int() with base 10: 'trois'
```

La solution. On place les instructions qui risquent de provoquer une erreur dans un bloc `try` (« essaie »). Python exécute ce bloc normalement. Si une erreur `ValueError` se produit, il abandonne immédiatement la suite du bloc `try` et saute dans le bloc `except ValueError`. Le programme ne plante plus : il continue après le `except`.

Exemple.

```
try:
    age = int(input("Ton âge ? "))
    print(f"Tu as {age} ans")
except ValueError:
    print("Ce n'est pas un nombre !")
```

Saisie	Ce qui se passe
L'utilisateur tape 15	`int()` réussit, le `print` du `try` affiche « Tu as 15 ans ». Le bloc `except` est ignoré.
L'utilisateur tape quinze	`int()` échoue : le `print` du `try` est sauté, et Python affiche « Ce n'est pas un nombre ! ».

Redemander jusqu'à ce que ce soit bon. Pour reposer la question en cas d'erreur, on met le `try` dans une boucle `while` contrôlée par une variable `ok` :

```
ok = False
while not ok:
    try:
        age = int(input("Ton âge ? "))
        ok = True
    except ValueError:
        print("Ce n'est pas un nombre, recommence !")
```

La ligne `ok = True` est placée juste après la conversion. Si la conversion échoue, Python saute directement dans le `except` : `ok` reste à `False` et la boucle recommence. Si elle réussit, `ok` passe à `True` et la boucle s'arrête.

Dans le jeu des allumettes, il y a deux raisons de redemander : le joueur n'a pas tapé un nombre (c'est le rôle du `except`), ou il a tapé un nombre interdit (c'est le rôle d'un `if` placé dans le `try`). La variable `ok` ne doit passer à `True` que si le nombre est valide **et** autorisé. Pense à afficher un message différent dans chaque cas.

Étape 5 : le tour de l'ordinateur

Sinon, c'est à l'ordinateur. Il prend `(allumettes - 1) % 4` allumettes. Si ce calcul donne 0, il en prend 1. Affiche combien il en prend.

Étape 6 : fin du tour

Retire les allumettes prises du tas, puis change de joueur. Astuce : `tour_joueur = not tour_joueur` inverse `True` et `False`.

Étape 7 : le gagnant

Après la boucle, le dernier à avoir joué a pris la dernière allumette, donc il a perdu. Mais attention, `tour_joueur` a déjà été inversé ! Si `tour_joueur` vaut `True`, qui a joué en dernier ? Affiche un message pour annoncer le résultat.

Squelette à compléter

Remplace chaque `...` par le bon code.

```
allumettes = 20
tour_joueur = True

while ... :
    ...
    if ... :
        ok = False
        while ... :
            try:
                prise = int(input("Combien en prends-tu (1 à 3) ? "))
                if ... :
                    ok = True
                else:
                    ...
            except ValueError:
                ...
    else:
        prise = ...
        if ... :
            prise = 1
        ...
    allumettes = ...
    tour_joueur = ...

if ... :
    ...
else:
    ...
```

Question bonus

L'ordinateur suit une stratégie. Combien d'allumettes le joueur doit-il prendre au premier tour pour être sûr de gagner ?

Le jeu des allumettes

Partie 1 : version console (ultra guidée)

Règle du jeu

Il y a **20 allumettes** sur la table. Le joueur et l'ordinateur jouent chacun leur tour, en commençant par le joueur. À chaque tour, on retire 1, 2 ou 3 allumettes. **Celui qui prend la dernière allumette a perdu.**

On construit le programme **petit à petit**. À chaque étape, tu lis l'explication, tu fais le petit exercice « À toi ! », puis tu testes avant de passer à la suite. Garde le même fichier du début à la fin.

Étape 1 : le tas d'allumettes

Une variable sert à retenir une valeur. On crée une variable `allumettes` qui vaut 20. Pour en retirer, on calcule la nouvelle valeur et on la range dans la même variable :

```
allumettes = 20
allumettes = allumettes - 3
print(f"Il reste {allumettes} allumettes")
```

La deuxième ligne se lit « la nouvelle valeur de `allumettes` est l'ancienne moins 3 ». Le programme affiche donc « Il reste 17 allumettes ».

À toi ! Écris ce programme, puis remplace le 3 par 2.

Teste : le programme doit afficher « Il reste 18 allumettes ».

Étape 2 : demander un nombre au joueur

`input()` attend que le joueur tape quelque chose au clavier. Comme ce qu'il tape est du texte, on le convertit en nombre entier avec `int()` :

```
prise = int(input("Combien en prends-tu (1, 2 ou 3) ? "))
allumettes = allumettes - prise
```

À toi ! Remplace la ligne `allumettes = allumettes - 2` par ces deux lignes.

Teste : tape 3, le programme doit afficher 17. Tape 1, il doit afficher 19.

Problème : si le joueur tape 10, le programme accepte et affiche « Il reste 10 allumettes ». C'est de la triche ! Les étapes 3 et 4 vont l'en empêcher.

Étape 3 : reconnaître un nombre interdit

Un nombre est interdit s'il est plus petit que 1 **ou** plus grand que 3. En Python, « ou » s'écrit `or` : la condition est vraie dès qu'**au moins une** des deux parties est vraie.

```
if prise < 1 or prise > 3:
    print("Interdit !")
else:
    print("Autorisé")
```

À toi ! Complète le tableau sur ta feuille, **sans ordinateur**, puis vérifie avec le programme.

prise	`prise < 1`	`prise > 3`	`prise < 1 or prise > 3`	**Résultat**
0	True	False	True	Interdit
2				
3				
5				
-1				

Autres écritures possibles, qui donnent le même résultat :

```
if prise != 1 and prise != 2 and prise != 3:  # ni 1, ni 2, ni 3
if not (1 <= prise <= 3):                      # PAS entre 1 et 3
```

Piège classique : écrire `prise < 1 and prise > 3`. Un nombre ne peut pas être à la fois plus petit que 1 **et** plus grand que 3, donc cette condition est toujours fausse : tout serait autorisé !

Étape 4 : obliger le joueur à recommencer

Avec un `if`, on ne repose la question qu'**une seule fois**. Si le joueur se trompe deux fois de suite, il triche quand même. Il faut une boucle `while` : tant que le nombre est interdit, on redemande.

```
prise = int(input("Combien en prends-tu (1, 2 ou 3) ? "))
while prise < 1 or prise > 3:
    print("Tu dois choisir 1, 2 ou 3 !")
    prise = int(input("Combien en prends-tu (1, 2 ou 3) ? "))
```

Voici ce qui se passe si le joueur tape 7, puis 0, puis 2 :

Saisie	**Condition du while**	**Que se passe-t-il ?**
7	`7 < 1 or 7 > 3` → True	on entre dans la boucle : message, et on redemande
0	`0 < 1 or 0 > 3` → True	on reste dans la boucle : message, et on redemande
2	`2 < 1 or 2 > 3` → False	on sort de la boucle : 2 est accepté

Piège classique : oublier la dernière ligne dans la boucle. Si on ne redemande pas, `prise` ne change jamais et la boucle tourne à l'infini (arrête le programme avec Ctrl+C).

À toi ! Ajoute cette boucle à ton programme.

Teste : tape 7, puis 0, puis 2. Le programme doit afficher deux messages d'erreur, puis « Il reste 18 allumettes ».

Étape 5 : ne pas prendre plus que ce qu'il reste

Quand il ne reste que 2 allumettes, prendre 3 est aussi interdit. On ajoute une troisième condition avec `or`. Pour afficher un message différent selon l'erreur, on utilise `if` / `elif` / `else` et une variable `ok` qui passe à `True` quand le choix est bon :

```
ok = False
while not ok:
    prise = int(input("Combien en prends-tu (1, 2 ou 3) ? "))
    if prise < 1 or prise > 3:
        print("Tu dois choisir 1, 2 ou 3 !")
    elif prise > allumettes:
        print(f"Il ne reste que {allumettes} allumettes !")
    else:
        ok = True
```

`while not ok` se lit « tant que ce n'est pas bon ». On ne sort de la boucle que quand on passe dans le `else`.

À toi ! Remplace ta boucle de l'étape 4 par celle-ci. Mets `allumettes = 2` au début pour tester.

Teste : tape 3 : le programme doit refuser. Tape 2 : il doit afficher « Il reste 0 allumettes ». Remets ensuite 20.

Étape 6 : résister aux lettres avec try / except

Si le joueur tape « deux », `int()` provoque une erreur `ValueError` et le programme plante. On met la saisie et les tests dans un bloc `try` : en cas d'erreur, Python saute dans le bloc `except`, `ok` reste à `False` et la boucle redemande.

```
ok = False
while not ok:
    try:
        prise = int(input("Combien en prends-tu (1, 2 ou 3) ? "))
        if prise < 1 or prise > 3:
            print("Tu dois choisir 1, 2 ou 3 !")
        elif prise > allumettes:
            print(f"Il ne reste que {allumettes} allumettes !")
        else:
            ok = True
    except ValueError:
        print("Ce n'est pas un nombre !")
```

À toi ! Ajoute le `try` / `except`. Attention à décaler (indenter) les lignes qui sont dans le `try`.

Teste : tape « deux », puis 5, puis 1. Le programme doit afficher deux messages différents, puis accepter 1.

Étape 7 : la boucle de jeu

Une partie, ce sont des tours qui se répètent **tant qu'il reste des allumettes**. Une variable `tour_joueur` retient à qui c'est de jouer : `True` pour le joueur, `False` pour l'ordinateur. À la fin de chaque tour, `tour_joueur = not tour_joueur` passe la main.

```
allumettes = 20
tour_joueur = True

while allumettes > 0:
    print(f"Il reste {allumettes} allumettes")
    if tour_joueur:
        # ici : ta boucle de saisie de l'étape 6
    else:
        # ici : le tour de l'ordinateur (étape 8)
    allumettes = allumettes - prise
    tour_joueur = not tour_joueur
```

À toi ! Organise ton programme comme ci-dessus et colle ta boucle de saisie à la bonne place, décalée de deux crans. Pour l'instant, mets `prise = 1` dans le `else`.

Teste : l'ordinateur doit prendre 1 allumette à chaque tour, et le jeu doit s'arrêter à 0.

Étape 8 : l'ordinateur joue malin

L'ordinateur utilise %, qui donne le **reste** d'une division : `17 % 4` vaut 1, car 17 = 4 × 4 + 1. Il prend `(allumettes - 1) % 4` allumettes, et 1 si le calcul donne 0.

```
prise = (allumettes - 1) % 4
if prise == 0:
    prise = 1
print(f"L'ordinateur en prend {prise}")
```

À toi ! Remplace `prise = 1` par ces lignes.

Teste : joue une partie. Tu vas sûrement perdre… c'est normal !

Étape 9 : annoncer le gagnant

Après la boucle, celui qui a joué en dernier a pris la dernière allumette : il a perdu. Mais `tour_joueur` a déjà été inversé. S'il vaut `True`, c'est donc l'**ordinateur** qui a joué en dernier.

```
if tour_joueur:
    print("L'ordinateur a pris la dernière : tu as gagné !")
else:
    print("Tu as pris la dernière : perdu !")
```

Piège classique : placer ces lignes **dans** la boucle de jeu. Elles doivent être après, sans décalage, au même niveau que le `while`.

À toi ! Ajoute ces lignes après la boucle.

Teste : le message doit s'afficher une seule fois, à la fin de la partie.

Étape 10 : rejouer une partie

Pour rejouer, on met **toute la partie** dans une deuxième boucle `while`. Une variable `rejouer` vaut 1 pour continuer et 0 pour arrêter.

```
rejouer = 1
while rejouer == 1:
    allumettes = 20
    tour_joueur = True
    # ici : la boucle de jeu (étape 7)
    # ici : l'annonce du gagnant (étape 9)
    rejouer = int(input("Tape 1 pour rejouer, 0 pour arrêter : "))

print("Merci d'avoir joué !")
```

Piège classique : laisser `allumettes = 20` **avant** la grande boucle. Au deuxième tour, il resterait 0 allumette et la nouvelle partie serait finie avant de commencer ! Tout ce qui doit être remis à zéro à chaque partie va **dans** la boucle.

Obliger à répondre 0 ou 1. C'est la même technique qu'à l'étape 4. Cette fois, la réponse est interdite si elle est différente de 0 **et** différente de 1 :

```
rejouer = int(input("Tape 1 pour rejouer, 0 pour arrêter : "))
while rejouer != 0 and rejouer != 1:
    rejouer = int(input("Réponds 1 ou 0 : "))
```

Pourquoi `and` ici et `or` à l'étape 3 ? Teste avec 1 : `1 != 0` est vrai, mais `1 != 1` est faux. Avec `or`, la condition serait toujours vraie, et la boucle ne s'arrêterait jamais.

À toi ! Décale tout ton jeu de quatre espaces pour le mettre dans la boucle `while rejouer == 1`, puis ajoute la question.

Teste : joue deux parties de suite, puis tape 5, puis 0. Le programme doit refuser 5, puis afficher « Merci d'avoir joué ! ».

Étape 11 : bonus : compter les victoires

Crée deux variables `victoires_joueur` et `victoires_ordi` qui valent 0. Ajoute 1 à la bonne variable quand quelqu'un gagne, et affiche le score après chaque partie.

À toi ! Où dois-tu créer ces deux variables : avant la grande boucle ou dedans ? Réfléchis au piège de l'étape 10 avant d'essayer.

Teste : après trois parties, le total des deux scores doit faire 3.

Le jeu des allumettes

Partie 2 : version graphique (brute)

Règle du jeu

Les règles sont les mêmes que dans la partie 1. L'ordinateur prend `(allumettes - 1) % 4` allumettes, ou 1 si ce calcul donne 0.

Travail demandé

Avec la bibliothèque `nsi_ui`, crée une interface graphique pour jouer au jeu des allumettes. Elle doit contenir :

- un texte qui affiche le nombre d'allumettes restantes ;
- trois boutons « Prendre 1 », « Prendre 2 » et « Prendre 3 », placés côte à côte ;
- un texte qui affiche les messages : ce que l'ordinateur a joué, un coup interdit, le gagnant.

Quand le joueur clique sur un bouton, le programme retire ses allumettes, puis l'ordinateur joue immédiatement. Une fois la partie terminée, les boutons ne doivent plus rien faire.

Le fichier `nsi_ui.py` doit se trouver dans le même dossier que ton programme.

Pour aller plus loin

- Ajoute un bouton « Rejouer » qui remet la partie à zéro.
- Ajoute un troisième label qui affiche le score : le nombre de victoires du joueur et de l'ordinateur.

Le jeu des allumettes

Partie 2 : version graphique (guidée)

Règle du jeu

Les règles sont les mêmes que dans la partie 1. L'ordinateur prend `(allumettes - 1) % 4` allumettes, ou 1 si ce calcul donne 0.

Ce qui change

En version graphique, il n'y a plus de boucle `while` pour faire tourner le jeu : c'est la programmation **réactive**. Le programme attend que le joueur clique sur un bouton, et c'est ce clic qui déclenche la fonction de rappel. Le tour de l'ordinateur se fait juste après, dans la même fonction.

Étape 1 : les variables

Crée une variable `allumettes` qui vaut 20, et une variable `fini` qui vaut `False` (elle indique si la partie est terminée).

Étape 2 : l'interface

Dans un conteneur vertical, crée un `label` nommé `reste`, puis un conteneur horizontal avec les trois boutons, puis un `label` nommé `message`. Termine par `main_loop()`.

Étape 3 : un bouton avec un paramètre

La fonction `button` accepte un troisième paramètre : `button("Prendre 2", jouer, 2)` appelle `jouer(2)` quand on clique. Les trois boutons utilisent donc la même fonction `jouer(prise)`.

Étape 4 : modifier une variable dans une fonction

Pour qu'une fonction puisse modifier une variable créée en dehors d'elle, on écrit `global allumettes, fini` sur la première ligne de la fonction. Sans cette ligne, Python provoque une erreur.

Étape 5 : les cas impossibles

Dans `jouer`, vérifie d'abord si la partie est finie, puis si le joueur prend plus d'allumettes qu'il n'en reste. Dans ces deux cas, affiche un message avec `set_text(message, ...)` et ne fais rien d'autre.

Étape 6 : le coup du joueur, puis celui de l'ordinateur

Sinon, retire les allumettes du joueur. S'il n'en reste plus, il a perdu : affiche-le et passe `fini` à `True`. Sinon, l'ordinateur joue avec sa stratégie, puis on vérifie de la même façon s'il a pris la dernière.

Étape 7 : mettre à jour l'affichage

À la fin de `jouer`, mets à jour le label `reste` avec une chaîne de formatage.

Squelette à compléter

Remplace chaque `...` par le bon code. Le fichier `nsi_ui.py` doit se trouver dans le même dossier.

```
from nsi_ui import *

allumettes = 20
fini = False

def jouer(prise):
    global allumettes, fini
    if ... :
        set_text(message, ...)
    elif ... :
        set_text(message, ...)
    else:
        allumettes = ...
        if ... :
            ...
            fini = ...
        else:
            ordi = ...
            if ... :
                ordi = 1
            allumettes = ...
            if ... :
                ...
                ...
            else:
                set_text(message, f"...")
    set_text(reste, f"...")

begin_vertical()
reste = label("Il reste 20 allumettes")
begin_horizontal()
button("Prendre 1", jouer, 1)
...
...
end_horizontal()
message = label("À toi de jouer !")
end_vertical()
main_loop()
```

Pour aller plus loin

- Ajoute un bouton « Rejouer » qui appelle une fonction `rejouer()` : elle remet `allumettes` à 20, `fini` à `False` et met à jour les deux labels (pense à `global` !).
- Ajoute un label `score` qui affiche le nombre de victoires de chacun, et mets-le à jour à la fin de chaque partie.

Le jeu des allumettes

Partie 2 : version graphique (ultra guidée)

Règle du jeu

Les règles sont les mêmes que dans la partie 1. L'ordinateur prend `(allumettes - 1) % 4` allumettes, ou 1 si ce calcul donne 0.

On construit l'interface **petit à petit**, dans le même fichier. À chaque étape, tu lances le programme pour vérifier que la fenêtre ressemble à ce qui est décrit dans « Teste ». Le fichier `nsi_ui.py` doit se trouver dans le même dossier que ton programme.

Étape 1 : ta première fenêtre

On importe la bibliothèque, on crée un `label` (une zone de texte), puis on lance l'interface avec `main_loop()`.

```
from nsi_ui import *

label("Bonjour !")
main_loop()
```

Piège classique : `main_loop()` doit toujours rester la **dernière ligne** du programme. Tout ce qui est écrit après ne sera exécuté qu'à la fermeture de la fenêtre.

À toi ! Écris ce programme et lance-le.

Teste : une petite fenêtre s'ouvre avec le texte « Bonjour ! ».

Étape 2 : afficher le nombre d'allumettes

On crée une variable `allumettes`, puis un label qui affiche sa valeur avec une chaîne de formatage. Cette fois, on **range le label dans une variable** `reste` : c'est grâce à elle qu'on pourra changer son texte plus tard avec `set_text`.

```
from nsi_ui import *

allumettes = 20
reste = label(f"Il reste {allumettes} allumettes")
main_loop()
```

À toi ! Remplace « Bonjour ! » par ce label, puis change la valeur de `allumettes` en 15.

Teste : la fenêtre affiche « Il reste 15 allumettes ». Remets 20 ensuite.

Étape 3 : un bouton et sa fonction de rappel

Un bouton est associé à une **fonction de rappel** : Python l'appelle à chaque clic. On écrit d'abord la fonction, puis on crée le bouton en lui donnant le **nom** de la fonction.

```
def jouer():
    set_text(reste, "Tu as cliqué !")

button("Prendre 1", jouer)
```

Place la fonction juste après `allumettes = 20`, et le bouton juste avant `main_loop()`. Un programme graphique suit toujours ce plan :

Ordre	Contenu
1	`from nsi_ui import *`
2	les variables du jeu
3	les fonctions de rappel (`def ...`)
4	l'interface : labels, boutons, conteneurs
5	`main_loop()`

Piège classique : écrire `button("Prendre 1", jouer())` avec des parenthèses. Python appellerait `jouer` une seule fois, au lancement, au lieu de l'appeler à chaque clic. On donne le nom de la fonction, **sans parenthèses**.

À toi ! Ajoute la fonction et le bouton.

Teste : clique sur le bouton : le texte devient « Tu as cliqué ! ».

Étape 4 : modifier une variable dans une fonction

Maintenant, chaque clic doit retirer une allumette. Mais une fonction n'a pas le droit de modifier une variable créée en dehors d'elle, sauf si on l'annonce avec le mot-clé `global` sur sa première ligne :

```
def jouer():
    global allumettes
    allumettes = allumettes - 1
    set_text(reste, f"Il reste {allumettes} allumettes")
```

Piège classique : oublier la ligne `global allumettes`. Au premier clic, Python affiche une erreur `UnboundLocalError` dans la console, et rien ne change dans la fenêtre.

À toi ! Modifie ta fonction `jouer`. Essaie d'abord sans la ligne `global` pour voir l'erreur, puis ajoute-la.

Teste : chaque clic fait baisser le nombre de 1 : 19, 18, 17…

Étape 5 : trois boutons, une seule fonction

Il faudrait trois fonctions presque identiques pour prendre 1, 2 ou 3 allumettes. Plutôt que cela, on donne un **paramètre** `prise` à `jouer`, et on utilise le troisième paramètre de `button` : `button("Prendre 2", jouer, 2)` appelle `jouer(2)` à chaque clic.

Pour placer les boutons côte à côte, on les met dans un conteneur horizontal, lui-même rangé dans un conteneur vertical :

```
def jouer(prise):
    global allumettes
    allumettes = allumettes - prise
    set_text(reste, f"Il reste {allumettes} allumettes")

begin_vertical()
reste = label(f"Il reste {allumettes} allumettes")
begin_horizontal()
button("Prendre 1", jouer, 1)
button("Prendre 2", jouer, 2)
button("Prendre 3", jouer, 3)
end_horizontal()
end_vertical()
main_loop()
```

Piège classique : oublier un `end_horizontal()` ou un `end_vertical()`. Chaque `begin_...` doit avoir son `end_...`, comme une parenthèse ouverte doit être fermée.

À toi ! Mets en place les trois boutons.

Teste : « Prendre 2 » fait passer de 20 à 18, puis « Prendre 3 » de 18 à 15.

Étape 6 : un label pour les messages et les coups interdits

Avec des boutons, le joueur ne peut choisir que 1, 2 ou 3 : pas besoin de vérifier ! Il reste un cas interdit : prendre plus que ce qu'il reste. On ajoute un label `message` sous les boutons, et un test dans `jouer` :

```
def jouer(prise):
    global allumettes
    if prise > allumettes:
        set_text(message, f"Il ne reste que {allumettes} allumettes !")
    else:
        allumettes = allumettes - prise
        set_text(message, f"Tu en as pris {prise}.")
    set_text(reste, f"Il reste {allumettes} allumettes")
```

Le label se crée dans l'interface, après `end_horizontal()` : `message = label("À toi de jouer !")`.

Comparaison avec la console : en console, il fallait une boucle `while` pour obliger le joueur à recommencer. Ici, on affiche simplement le message d'erreur et on ne fait rien d'autre : le programme attend tout seul le prochain clic. C'est la programmation **réactive**.

À toi ! Ajoute le label et le test.

Teste : clique jusqu'à ce qu'il reste 2 allumettes, puis sur « Prendre 3 » : le message d'erreur s'affiche et le nombre ne change pas.

Étape 7 : l'ordinateur répond

En console, une boucle faisait jouer le joueur puis l'ordinateur. Ici, il n'y a pas de boucle : l'ordinateur joue **juste après** le joueur, dans la même fonction. On remplace la ligne `set_text(message, f"Tu en as pris {prise}.")` par :

```
        ordi = (allumettes - 1) % 4
        if ordi == 0:
            ordi = 1
        allumettes = allumettes - ordi
        set_text(message, f"L'ordinateur en a pris {ordi}. À toi !")
```

`%` donne le reste de la division : `17 % 4` vaut 1, car 17 = 4 × 4 + 1.

À toi ! Ajoute le tour de l'ordinateur, bien décalé dans le else.

Teste : après « Prendre 1 », l'ordinateur en prend 2 et il reste 17 allumettes.

Étape 8 : détecter la fin de la partie

Il y a deux moments où la partie peut se finir : juste après le coup du joueur, et juste après celui de l'ordinateur. À chaque fois, on teste s'il reste 0 allumette. Une variable `fini` retient que la partie est terminée.

```
allumettes = 20
fini = False

def jouer(prise):
    global allumettes, fini
    if fini:
        set_text(message, "Partie terminée !")
    elif prise > allumettes:
        set_text(message, f"Il ne reste que {allumettes} allumettes !")
    else:
        allumettes = allumettes - prise
        if allumettes == 0:
            set_text(message, "Tu as pris la dernière : perdu !")
            fini = True
        else:
            ordi = (allumettes - 1) % 4
            if ordi == 0:
                ordi = 1
            allumettes = allumettes - ordi
            if allumettes == 0:
                set_text(message, "Tu as gagné !")
                fini = True
            else:
                set_text(message, f"L'ordinateur en a pris {ordi}.")
    set_text(reste, f"Il reste {allumettes} allumettes")
```

Le premier test `if fini:` empêche de continuer à cliquer quand la partie est terminée. Remarque que `global` accepte plusieurs variables, séparées par des virgules.

Piège classique : tester `allumettes == 0` seulement à la fin de la fonction. Si le joueur prend la dernière, l'ordinateur jouerait quand même et le nombre deviendrait négatif !

À toi ! Complète ta fonction.

Teste : joue une partie jusqu'au bout. Le bon message s'affiche, puis les boutons ne font plus rien.

Étape 9 : rejouer une partie

Pour rejouer, on ajoute un bouton « Rejouer » qui appelle une fonction `rejouer()`. Elle remet les variables à leur valeur de départ, puis met à jour l'affichage :

```
def rejouer():
    global allumettes, fini
    allumettes = 20
    fini = False
    set_text(reste, f"Il reste {allumettes} allumettes")
    set_text(message, "Nouvelle partie : à toi de jouer !")
```

Le bouton se crée dans l'interface, sous le label `message` : `button("Rejouer", rejouer)`.

Comparaison avec la console : en console, il fallait mettre toute la partie dans une grande boucle `while rejouer == 1`. Ici, il suffit de **remettre les variables à zéro** : le programme attend déjà les clics.

Piège classique : oublier `global` dans `rejouer`, ou oublier les `set_text`. Dans le premier cas, les variables ne sont pas remises à zéro. Dans le second, elles le sont, mais la fenêtre affiche toujours l'ancien texte.

À toi ! Ajoute la fonction et le bouton.

Teste : termine une partie, clique sur « Rejouer » : il reste 20 allumettes et les boutons fonctionnent à nouveau.

Étape 10 : bonus : afficher le score

Crée deux variables `victoires_joueur` et `victoires_ordi` qui valent 0, et un label `score` sous le bouton « Rejouer ». Dans `jouer`, ajoute 1 à la bonne variable quand quelqu'un gagne, et mets à jour le label.

À toi ! Faut-il remettre les victoires à 0 dans `rejouer()` ? Quelles variables faut-il ajouter à la ligne `global` de `jouer` ?

Teste : après deux parties, le score affiche bien deux victoires au total.

Pour aller encore plus loin : la partie 3 remplace les trois boutons par un champ de saisie, et utilise `try` / `except` pour résister aux lettres.

Le jeu des allumettes

Partie 3 : variante avec champ de saisie et try / except

Consigne

Reprends ta version graphique de la partie 2. Remplace les trois boutons « Prendre 1/2/3 » par un **champ de saisie** où le joueur tape le nombre d'allumettes qu'il prend, et un bouton « Jouer ». Le programme ne doit jamais planter, même si le joueur tape n'importe quoi.

Étape 1 : d'où vient l'erreur ?

La fonction `get_int(champ)` lit le texte tapé dans le champ et le convertit en entier avec `int()`. Si le joueur tape « trois », la conversion échoue et provoque une erreur `ValueError`, exactement comme `int(input())` dans la partie 1.

Étape 2 : une nouvelle fonction de rappel

Le bouton « Jouer » appelle une fonction `clic_jouer()`, sans paramètre. Elle lit la valeur du champ dans un bloc `try`, vérifie qu'elle est entre 1 et 3, puis appelle `jouer(prise)`. La fonction `jouer` de la partie 2 ne change pas : elle vérifie déjà s'il reste assez d'allumettes.

Étape 3 : afficher les erreurs

Si ce n'est pas un nombre, le bloc `except ValueError` affiche un message dans le label `message`. Si c'est un nombre hors de 1 à 3, un `if` affiche un autre message.

Étape 4 : pourquoi plus de boucle while ?

Dans la partie 1, il fallait une boucle `while not ok` pour redemander la saisie. Ici, c'est inutile : on affiche simplement l'erreur, et le programme attend le prochain clic. Le joueur corrige sa saisie et reclique. C'est tout l'intérêt de la programmation **réactive**.

Squelette à compléter

Garde les variables et la fonction `jouer` de la partie 2, puis remplace l'interface par ce code.

```
def clic_jouer():
    try:
        prise = ...
        if ... :
            set_text(message, ...)
        else:
            ...
    except ValueError:
        ...

begin_vertical()
reste = label("Il reste 20 allumettes")
champ = entry("Combien en prends-tu (1 à 3) ?")
button("Jouer", ...)
message = label("À toi de jouer !")
end_vertical()
main_loop()
```

Pour aller plus loin

- Après chaque coup, vide le champ de saisie avec `set_text(champ, "")`.
- Fais en sorte que la touche Entrée joue aussi : `entry` accepte une fonction en deuxième paramètre, appelée avec le texte tapé quand on appuie sur Entrée.